

SAP ABAP – Unit 4

Exercise 13: Analyze and Use a CDS View Entity

Practical Exercise Notes • Step-by-step practice + solution

1. Exercise Objective

In Exercise 13, the local class is changed so that it reads connection information from the CDS view entity /DMO/I_Connection instead of reading directly from the database table /DMO/CONNECTION. The CDS view also exposes an association to the airline, allowing the airline name to be read through _Airline-Name.

2. What You Learn

- How to inspect a CDS view entity in Eclipse.
- How CDS element aliases differ from the underlying database field names.
- How to use a CDS association in an ABAP SQL SELECT.
- How to add a new private instance attribute to a local class.
- How to replace a database-table SELECT with a CDS-view SELECT.
- How to map SELECT results into multiple class attributes.
- Why the client field is not explicitly added to the WHERE condition for this CDS view.

3. Exercise Setup

Template: /LRN/CL_S4D400_DBS_SELECT

Solution: /LRN/CL_S4D400_DBS_CDS

Suggested personal class: ZCL_##_CDS

If you completed Exercise 12, you can continue from your existing class rather than copying the template again.

4. Task 1 – Copy the Template

Optional if Exercise 12 was completed.

1. Open /LRN/CL_S4D400_DBS_SELECT in Eclipse.
2. In Project Explorer, right-click the class and choose Duplicate.
3. Choose your package.
4. Use a name such as ZCL_##_CDS.
5. Adjust the description, confirm the transport request, and finish.

5. Task 2 – Analyze the CDS View Entity

Open the development object /DMO/I_Connection.

In Eclipse, use Ctrl + Shift + A, search for /DMO/I_Con, then open the CDS Data Definition.

5.1 Find the aliases

The CDS view is based on /dmo/connection, but the exposed element names are not necessarily the same as the database field names.

```
Connection.airport_from_id as DepartureAirport,  
Connection.airport_to_id   as DestinationAirport,
```

Therefore:

- airport_from_id → DepartureAirport
- airport_to_id → DestinationAirport

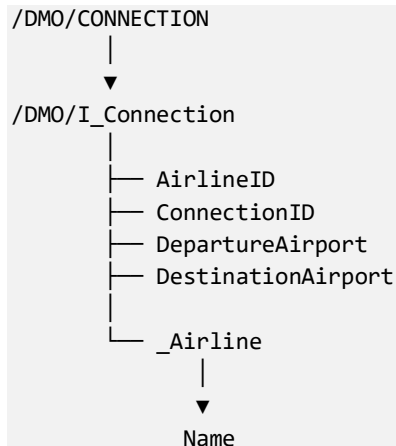
5.2 Analyze the _Airline association

```
association [1..1] to /DMO/I_Carrier as _Airline ...
```

The association points to the carrier CDS view. The carrier element described as "Flight Reference Scenario: Carrier Name" is:

- Element: Name
- Element type: /DMO/CARRIER_NAME

6. CDS View Structure – The Important Idea



The key advantage here is that the CDS view gives the ABAP program a business-oriented data model. Instead of directly reading a raw database field and separately looking up the carrier name, the SELECT can navigate through the CDS association.

7. Task 3 – Add carrier_name

Add carrier_name to the PRIVATE SECTION of lcl_connection. It is an instance attribute, so every connection object can hold its own airline name.

```
PRIVATE SECTION.  
  
DATA carrier_id      TYPE /dmo/carrier_id.  
DATA connection_id  TYPE /dmo/connection_id.  
DATA airport_from_id TYPE /dmo/airport_from_id.  
DATA airport_to_id   TYPE /dmo/airport_to_id.  
DATA carrier_name    TYPE /dmo/carrier_name.
```

Why private?

- The caller should not directly modify the object's internal data.
- The constructor controls how the values are initialized.
- `get_output( )` exposes the information in a controlled way.

7.1 Update `get_output( )`

Replace the carrier output line with:

```
APPEND |Carrier: { carrier_id } { carrier_name }| TO r_output.
```

At this stage `carrier_name` is not yet filled by the `SELECT`. It will become populated in Task 4.

8. Task 4 – Replace the Database `SELECT` with a CDS `SELECT`

Exercise 12 read directly from `/DMO/CONNECTION`. In Exercise 13, that `SELECT` is commented out and replaced by a `SELECT` from `/DMO/I_Connection`.

8.1 Keep the old `SELECT` as commented learning reference

```
* SELECT SINGLE
* FROM /dmo/connection
* FIELDS airport_from_id, airport_to_id
* WHERE carrier_id = @i_carrier_id
*       AND connection_id = @i_connection_id
* INTO ( @airport_from_id, @airport_to_id ).
```

8.2 New CDS `SELECT`

```
SELECT SINGLE
  FROM /DMO/I_Connection
  FIELDS DepartureAirport,
          DestinationAirport,
          \_Airline-Name
  WHERE AirlineID = @i_carrier_id
        AND ConnectionID = @i_connection_id
  INTO ( @airport_from_id,
         @airport_to_id,
         @carrier_name ).
```

9. Understand the `SELECT` Line by Line

Code	Meaning
<code>/DMO/I_Connection</code>	The CDS view entity being read.
<code>SELECT SINGLE</code>	Read one matching record.
<code>DepartureAirport</code>	CDS element containing the departure airport.
<code>DestinationAirport</code>	CDS element containing the destination airport.
<code>_Airline-Name</code>	Navigate through the <code>_Airline</code> association and read Name.
<code>AirlineID = @i_carrier_id</code>	Match the CDS carrier key to the constructor parameter.
<code>ConnectionID = @i_connection_id</code>	Match the CDS connection key to the constructor parameter.
<code>@airport_from_id</code>	Receive <code>DepartureAirport</code> .
<code>@airport_to_id</code>	Receive <code>DestinationAirport</code> .
<code>@carrier_name</code>	Receive the airline Name.

10. Why the Backslash Before _Airline?

In this ABAP SQL/CDS navigation syntax, the association is referenced as a CDS path. The exercise uses:

```
\_Airline-Name
```

This means: follow the _Airline association, then take its Name element. It is conceptually similar to navigating from a connection to its related carrier.

11. Why @ Is Used

```
WHERE AirlineID = @i_carrier_id
      AND ConnectionID = @i_connection_id

INTO ( @airport_from_id,
      @airport_to_id,
      @carrier_name ).
```

The @ distinguishes ABAP variables/host variables from database or CDS elements. For example, AirlineID is a CDS element, while i_carrier_id is an ABAP method parameter.

12. Why There Is No Client Field

The CDS view's element list does not expose the client field. Therefore you do not add a client condition such as MANDT = Client handling is managed by the ABAP/CDS framework for this scenario.

13. Error Handling

```
IF sy-subrc <> 0.
  RAISE EXCEPTION TYPE cx_abap_invalid_value.
ENDIF.
```

After SELECT SINGLE, sy-subrc indicates whether the database/CDS read succeeded. If no matching record is found, the constructor raises CX_ABAP_INVALID_VALUE instead of silently creating an object with missing airport or airline data.

14. Complete Local Class Code

```
CLASS lcl_connection DEFINITION.

  PUBLIC SECTION.

    CLASS-DATA conn_counter TYPE i READ-ONLY.

    METHODS constructor
      IMPORTING
        i_carrier_id TYPE /dmo/carrier_id
        i_connection_id TYPE /dmo/connection_id
      RAISING
        cx_abap_invalid_value.

    METHODS get_output
      RETURNING
        VALUE(r_output) TYPE string_table.
```

PROTECTED SECTION.

PRIVATE SECTION.

```
DATA carrier_id      TYPE /dmo/carrier_id.
DATA connection_id   TYPE /dmo/connection_id.
DATA airport_from_id TYPE /dmo/airport_from_id.
DATA airport_to_id    TYPE /dmo/airport_to_id.
DATA carrier_name     TYPE /dmo/carrier_name.
```

ENDCLASS.

CLASS lcl_connection IMPLEMENTATION.

METHOD constructor.

```
IF i_carrier_id IS INITIAL OR i_connection_id IS INITIAL.
  RAISE EXCEPTION TYPE cx_abap_invalid_value.
ENDIF.
```

```
* Old Exercise 12 SELECT
* SELECT SINGLE
* FROM /dmo/connection
* FIELDS airport_from_id, airport_to_id
* WHERE carrier_id = @i_carrier_id
*       AND connection_id = @i_connection_id
* INTO ( @airport_from_id, @airport_to_id ).
```

```
SELECT SINGLE
FROM /DMO/I_Connection
FIELDS DepartureAirport,
        DestinationAirport,
        \_Airline-Name
WHERE AirlineID = @i_carrier_id
      AND ConnectionID = @i_connection_id
INTO ( @airport_from_id,
        @airport_to_id,
        @carrier_name ).
```

```
IF sy-subrc <> 0.
  RAISE EXCEPTION TYPE cx_abap_invalid_value.
ENDIF.
```

```
me->carrier_id = i_carrier_id.
me->connection_id = i_connection_id.
```

```
conn_counter = conn_counter + 1.
```

ENDMETHOD.

METHOD get_output.

```
APPEND |-----| TO r_output.
APPEND |Carrier: { carrier_id } { carrier_name }| TO r_output.
APPEND |Connection: { connection_id }| TO r_output.
```

```

    APPEND |Departure: { airport_from_id }| TO r_output.
    APPEND |Destination: { airport_to_id }| TO r_output.

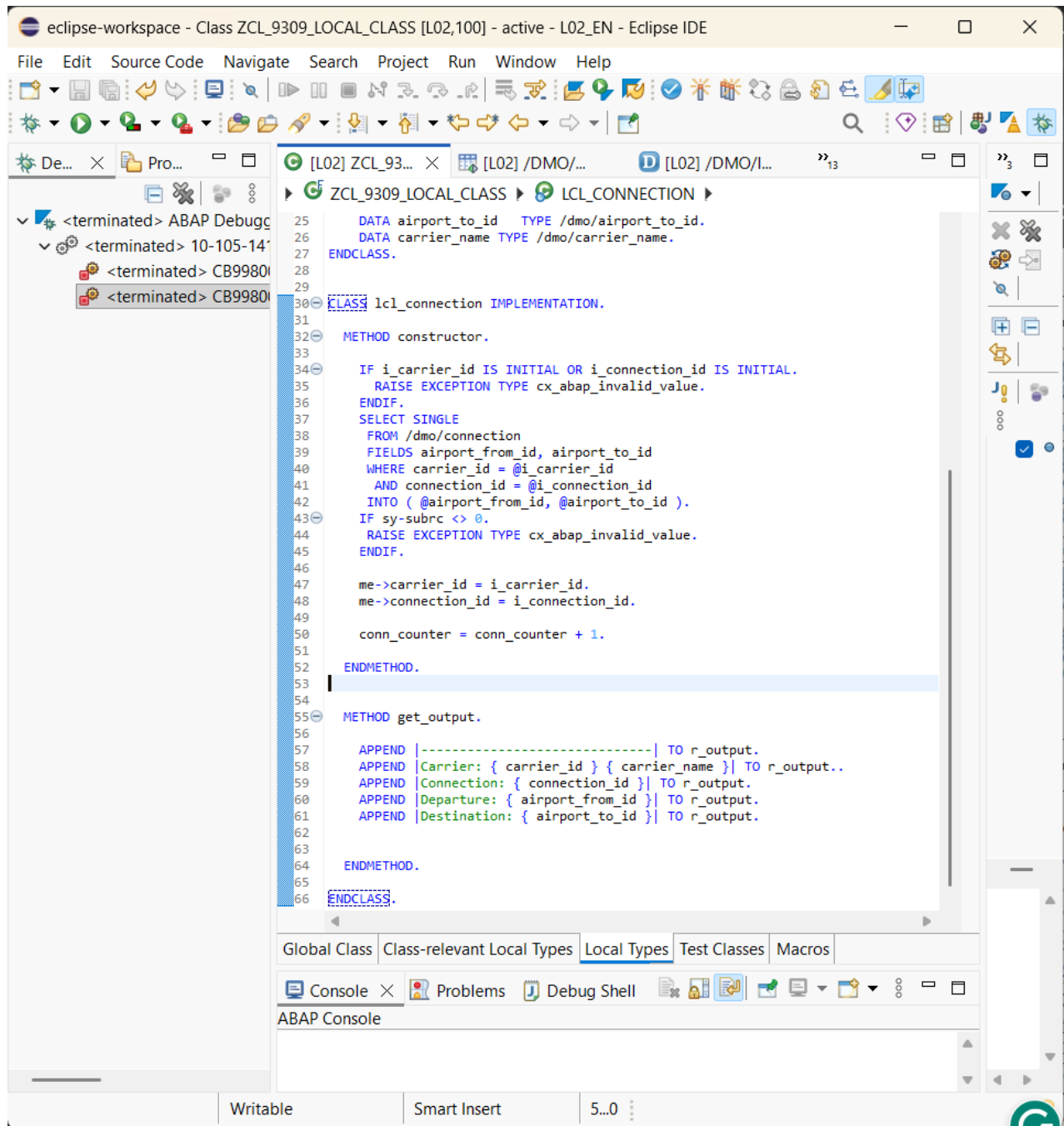
ENDMETHOD.

ENDCLASS.

```

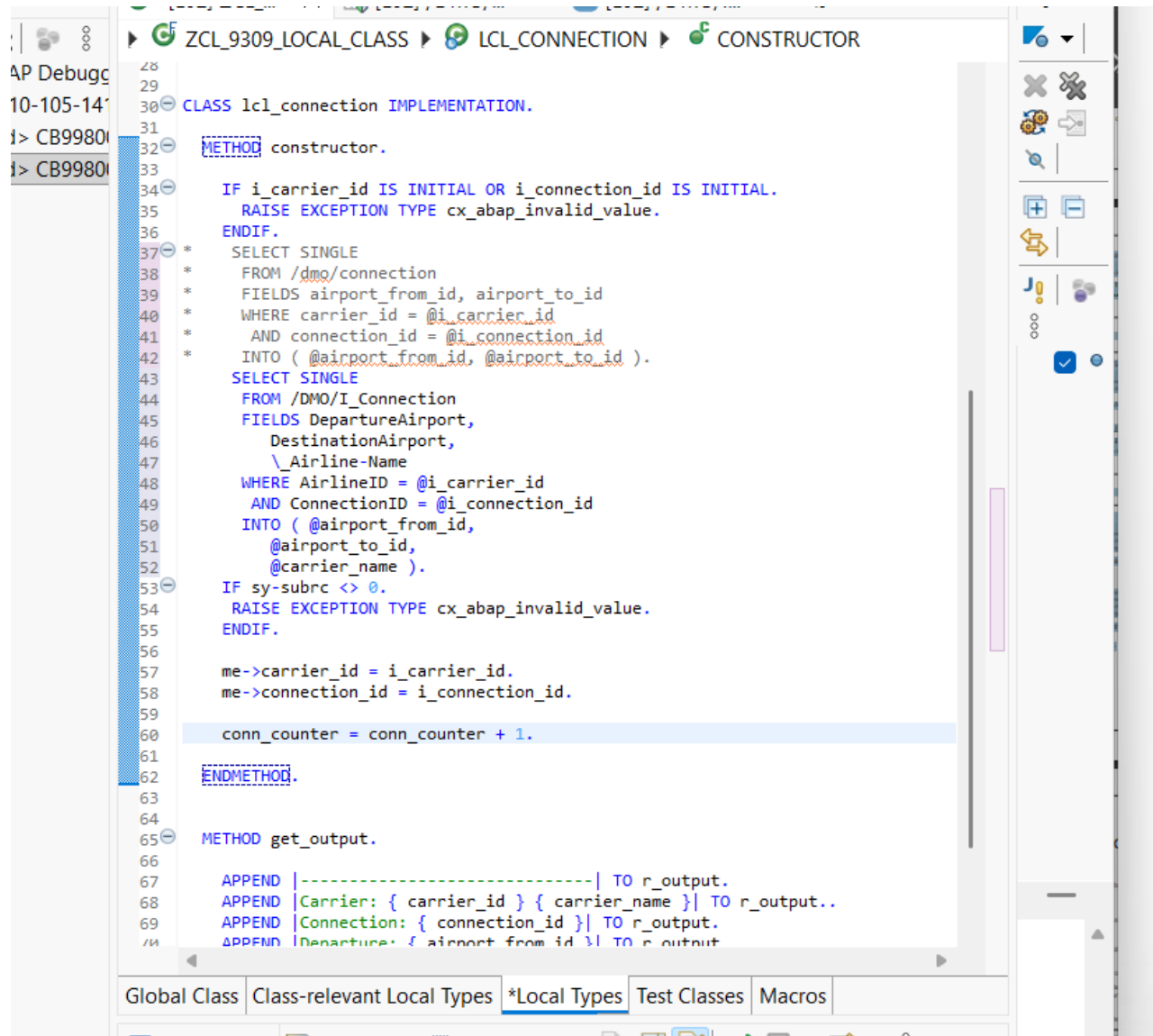
Note: The exact capitalization of ABAP identifiers is not important to the compiler, but keeping the spelling consistent with the exercise makes the code easier to compare.

15. Screenshot – Your CDS View Analysis



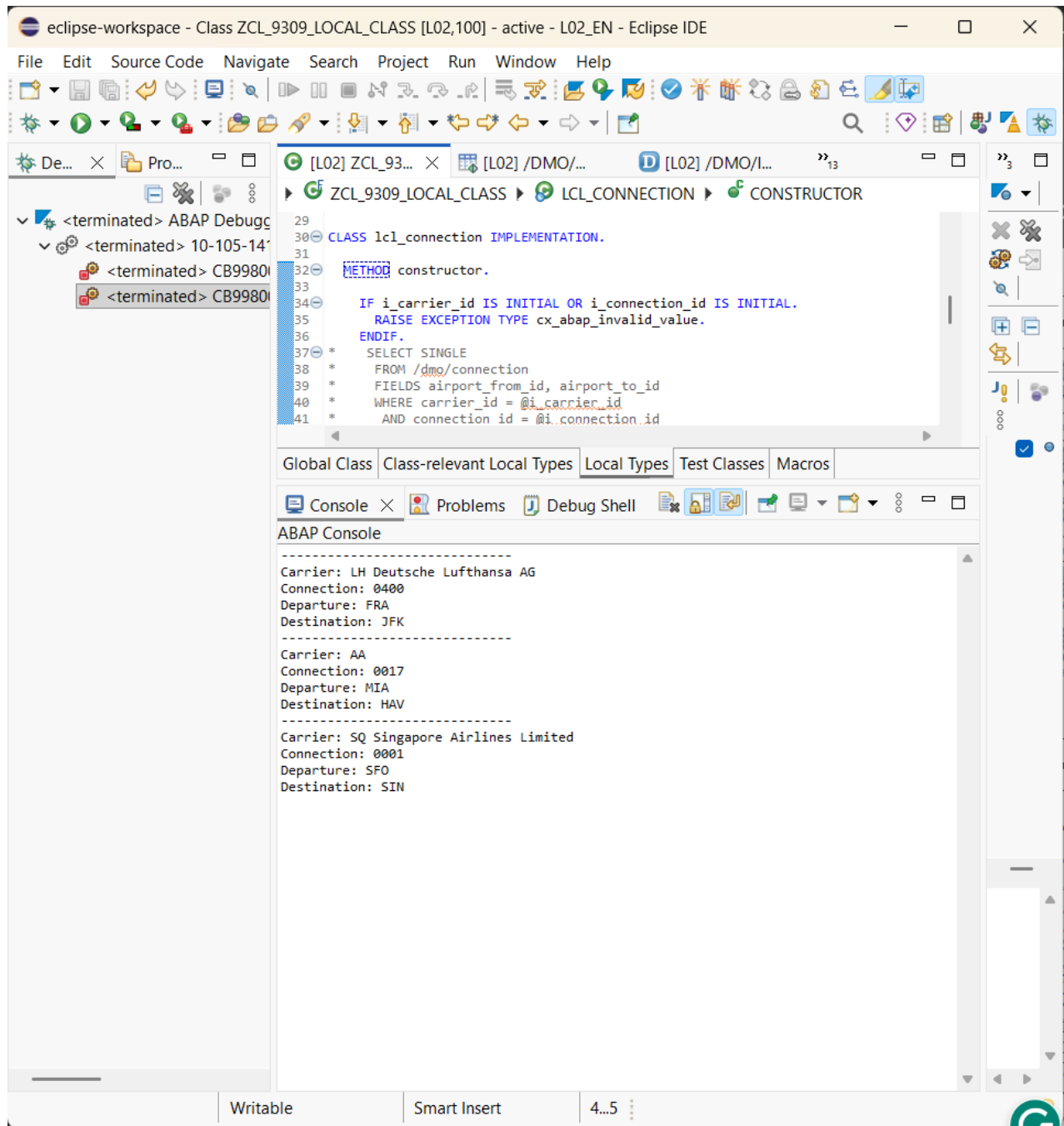
CDS view /DMO/I_Connection showing aliases such as DepartureAirport and DestinationAirport.

16. Screenshot – Your Constructor Implementation



Constructor with the CDS SELECT, association navigation, WHERE condition, and INTO mapping.

17. Screenshot – Successful Execution



Successful ABAP Console output showing airline names, connections, departure airports, and destinations.

18. Expected Console Output

```
Carrier: LH Deutsche Lufthansa AG  
Connection: 0400  
Departure: FRA  
Destination: JFK
```

```
Carrier: AA  
Connection: 0017  
Departure: MIA
```

Destination: HAV

Carrier: SQ Singapore Airlines Limited

Connection: 0001

Departure: SFO

Destination: SIN

19. How to Verify the Exercise

1. Check that carrier_name exists in the PRIVATE SECTION.
2. Check that get_output() prints carrier_name.
3. Check that the old /dmo/connection SELECT is commented out.
4. Check that the new SELECT uses /DMO/I_Connection.
5. Check that DepartureAirport and DestinationAirport are selected.
6. Check that _Airline-Name is selected.
7. Check that AirlineID and ConnectionID are used in WHERE.
8. Check that the three SELECT results map to airport_from_id, airport_to_id, and carrier_name.
9. Activate with Ctrl + F3.
10. Run with F9.
11. Confirm that airline names and airport values appear in the ABAP Console.

20. Exercise 12 vs Exercise 13

Topic	Exercise 12	Exercise 13
Read from	/DMO/CONNECTION	/DMO/I_Connection
Airport fields	airport_from_id / airport_to_id	DepartureAirport / DestinationAirport
Airline name	Not read	Read through _Airline-Name
Data model	Database table	CDS view entity
Association	No CDS association	Airline
Purpose	Direct table access	Business-oriented CDS access

21. Key Takeaway

The main lesson is not just the SELECT syntax. It is the architectural difference between reading a raw database table and reading a CDS view entity. A CDS view can expose meaningful business names, associations, and reusable data-model logic. In Exercise 13, /DMO/I_Connection lets the ABAP program read airport information and navigate to the airline name in one SELECT.

22. Exercise Status

Exercise 13: COMPLETED

Your successful console output confirms that the CDS SELECT, association navigation, attribute mapping, activation, and execution are working.